

*สัญลักษณ์ Diagram หน้า 2

I. Net-Beans

- (สำคัญ) วิธีการสร้าง Project Java ใน NetBeans
 - Categories --> Java with Ant
 - Projects --> Java Application

II. Basic Java and Variables Type

- การแสดงผลใน Terminal

System.out.println("");	
print	println
อยู่บรรทัดเดิม	ขึ้นบรรทัดใหม่

- การรับค่าผ่าน keyboard
 - นำเข้า Package Scanner

```
import java.util.Scanner;
```

- การสร้าง Object (ต่อเชื่อม)

```
Scanner sc = new Scanner(System.in);
```

- การรับค่าของข้อมูลแต่ละชนิด

```
String str = sc.nextLine();
int num = sc.nextInt();
Float num = sc.nextFloat();
double num = sc.nextDouble();
char c = sc.next().charAt(0);
```

- การประกาศค่าตัวแปรปกติ

```
int brainCells = 84000;
```

- การประกาศค่าของตัวแปรเป็น final (เปลี่ยนค่าได้ครั้งเดียว ไม่สามารถเปลี่ยนค่าได้แต่สามารถเรียกใช้ได้)

```
final int brainCells = 84000;
```

- ตัวดำเนินการทางคณิตศาสตร์แบบย่อ

แบบเต็ม	แบบย่อ	แบบลัด
a = a + 1;	a += 3;	a++ or ++a
a = a - 1;	a -= 1;	a-- or --a
a = a * 1;	a *= a;	-
a = a / 1;	a /= 1;	-
a = a % 1;	a %= 1;	-

III. ตัวดำเนินการ (Operators)

Input1		Input2	Output
int	+ -	int	int
int	* / %	double	double
String		String	String
int	+	String	String
double		String	String
char	>	char	boolean
int	>=	int	
double	<	double	
char	<=	char	
int	==	int	
double	!=	double	
boolean		boolean	

Logic Operator	
and	&& or &
or	or

- วิธีการเปรียบเทียบ String กับ String
 - boolean --> String1.equals(String2)

IV. เงื่อนไข (Condition)

```
if (condition) {}
else if (condition) {}
else {}
```

V. การทำซ้ำ (Iteration)

- For --> ทำซ้ำแบบรู้จำนวนรอบ
- While --> ทำซ้ำแบบใช้เงื่อนไข

```
for ( start ; stop ; step){
    // command
}

while (condition) {
    // command
}
```

- Break --> หยุดลูป
- Continue --> เริ่มต้นลูปใหม่

```
While True {
    If (condition) {
        continue; or break;
    }
    //code here
}
```

VI. การแปลงชนิดข้อมูล

- เครื่องหมาย "="

```
int a = 10;
double b = a;
```

- ตัวดำเนินการ Operator

```
int a = 10;
double b = 15.0;
System.out.println("Line 1: "+(a+b))
```

- วิธีการ Casting

```
double a = 3.14;
int b = (int) a;
```

- เมธอดจากคลาส Wrapper

```
String text1 = "12345";
Int num = Integer.parseInt(str);
```

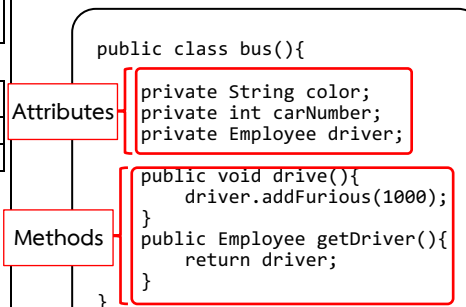
int --> String	String.valueOf(i);
String --> int	Integer.parseInt("200");
double --> String	String.valueOf(num2);
String --> double	Double.parseDouble("3.14");

--- OOP ---

VII. OOP

- ส่วนประกอบของคลาส

- Attribute — สิ่งทีบงบอกลักษณะต่างๆของ คลาส เช่น color, carNumber, busMan;
- Method — สิ่งทีอธิบายการทำงานของวัตถุใน คลาส เช่น drive(), emergencyBreak(), newDestination("Happy Land");



- Modifier --> Public, Private, Protected

- การสร้างคลาสใน Java

```
[modifier] class <classname> {}
EX: public class Bus{}
```

- การสร้างวัตถุจากคลาสใน Java

- จากตัวอย่างคือการสร้าง object มาจากคลาส Bus

```
Class <name> = new Class();
EX: Bus bus145 = new Bus();
    Bus bus3_34 = new Bus("Bangna");
```

VIII. Encapsulation (การห่อหุ้ม)

- เป็นวิธีการปกป้องข้อมูลจากการเข้าถึงจากภายนอกหรือโปรแกรมอื่น ๆ

- Access Modifier

Public (+)	ทุกคลาส
Private (-)	ภายในคลาสเท่านั้น
Protected (#)	คลาสแม่-ลูก
Default	คลาสใน package เดียวกัน

- Setter — ใช้กำหนดค่า attribute

- Getter — ใช้เรียกค่า attribute

IX. Inheritance (การสืบทอด)

- เป็นการสืบทอด attribute หรือ method มาจาก คลาสอื่นๆ โดยที่ทั้งคู่งต้องไม่กำหนด access modifier เป็น private
- คลาสแม่มี ลูกก็มี
- คลาสแม่มีลูกได้หลายคน แต่คลาสลูกมีแม่ได้คน เดียวเท่านั้น (Diamond Problem)

```
public class EVBus extends Bus{}
```

X. Polymorphism (การมีหลายรูปแบบ)

คุณสมบัติของอ็อบเจกต์ของคลาสที่ต่างกันสามารถ ตอบสนองต่อเมธอดเดียวกันในวิธีการที่ต่างกันได้ ซึ่ง หมายถึงการเขียนเมธอดแบบ Overloaded, Overridden และการใช้ Dynamic Binding

- Overloaded — เป็นการกำหนดให้เมธอด (ที่มี ชื่อเหมือนกัน) ภายในคลาสเดียวกัน โดยจะมีการ ทำงาน, อินพุต, ผลลัพธ์ที่แตกต่างกัน โดยสามารถ ทำการ overloaded ได้ 2 แนวทางคือ (1)เปลี่ยน จำนวนของ parameter และ (2)เปลี่ยนชนิดของ ข้อมูลใน parameter

```
public void drive(){}
public void drive(String driverName){}
public void drive(char driverName){}
public void drive(int driverID){}
public void drive(int driverID, String driverName){}
```

- !!! ชนิดของ parameter ต้องไม่เหมือนกัน !!!

```
public void drive(String driverName){}
public void drive(String destination){}

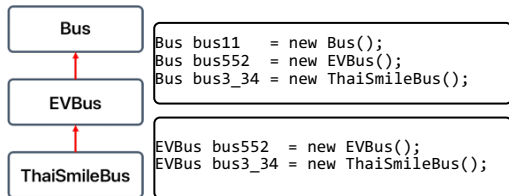
public void drive(int driverID){}
public void drive(int busNumber){}
```

- Overridden — เมธอดในคลาสลูกมีชื่อ เหมือนกับของคลาสแม่เพื่อให้คลาสลูกเพิ่มเติม การทำงานเฉพาะของตัวเอง

```
public class Bus{
    public void start(){
        System.out.println("honk honk");
    }
}

public class EVBus extends Bus{
    public void start(){
        System.out.println(" ");
    }
}
```

3. **Dynamic Blinding** — เป็นหลักการของการสร้างวัตถุของคลาสที่มีการสืบทอดกันได้หลากหลายรูปแบบ ตัวอย่างการประกาศให้วัตถุของคลาสใดๆ (ประกาศคลาสแม่ สร้างคลาสลูก)



4. **Virtual Method Invocation** — คอมไพเลอร์ของภาษาจาวาจะไม่อนุญาตให้เมธอดแม่เรียกใช้เมธอดของลูก

```
Bus bus3_34 = new Bus();
bus3_34.chargeBattery(); // Error
```

จากตัวอย่างข้างต้นในคลาส Bus จะไม่มีเมธอด chargeBattery(); เพราะว่าเมธอดนี้จะมีในคลาสลูก ชื่อ EVBus เราจึงต้องทำการ Casting Object หรือที่อาจารย์แบงก์บอกว่าเป็นการหลอกตัวคอมไพเลอร์

```
Bus bus3_34 = new Bus();
((EVBus)bus3_34).chargeBattery();
```

5. **Upcasting** — (การประกาศแม่แล้วสร้างลูก) คือการแปลงชนิดข้อมูลรูปแบบหนึ่งที่พยายามแปลงออปเจ็คจากคลาสลูกไปเป็นออปเจ็คของคลาสแม่ นิยมนำมาใช้เมื่อต้องการเข้าถึง attribute หรือ method ของคลาสลูก
6. **Downcasting** — (เอาแม่มาเป็นลูก) คือการแปลงชนิดข้อมูล (typecasting) รูปแบบหนึ่งที่พยายามแปลงออปเจ็คจากคลาสแม่ไปเป็นออปเจ็คของคลาสลูกซึ่งการทำ downcasting จะไม่เกิดข้อผิดพลาดใน Compile-Time

```
public class Bus{...}
public class EVBus extends Bus{...}
public class Main{
    public static void main(){
        // Upcasting
        Bus bus3_34 = new EVBus();

        // Downcasting
        Bus bus145 = new Bus();
        EVBus bus3_34 = (EVBus) bus145;
    }
}
```

7. **Instanceof** — คือตัวดำเนินการที่ใช้กับอ็อบเจกต์และคลาส เพื่อตรวจสอบว่าเป็นอ็อบเจกต์ของคลาสนั้นหรือไม่ ซึ่งจะให้ผลลัพธ์เป็นข้อมูลชนิด boolean โดยถ้าผลลัพธ์เป็น true แสดงว่าอ็อบเจกต์เป็นของคลาสนั้นหรือเป็นของคลาสที่คลาสนั้นสืบทอดมา

```
EVBus bus8 = new EVBus();

(bus8 instanceof EVBus)    --> True
(bus8 instanceof Bus)      --> True
(bus8 instanceof Object)   --> True
(bus8 instanceof ThaiSmilebus) --> False
(bus8 instanceof String)   --> Error!
```

8. **Dependency Injection** — (รอรับการขยายในอนาคต) สิ่งนี้ทำให้ทั้งสองคลาสลดการผูกมัดกันลงและทำให้สะดวกต่อการทดสอบการทำงานของแต่ละส่วน

XI. Static

ใช้สำหรับการกำหนด (1) attribute และ (2) เมธอด (method) ที่ใช้งานร่วมกันในทุก object ซึ่งสามารถเรียกใช้งานได้โดยไม่ต้องสร้างวัตถุมาก่อน

```
public class Main(){
    public static void main(){
        int a = Math.round(3.14);
        int b = String.valueOf(1221);
    }
}
```

สัญลักษณ์	ความหมาย
+	เป็นการกำหนด access mode เป็น public
-	เป็นการกำหนด access mode เป็น private
#	เป็นการกำหนด access mode เป็น protected
Class	บ่งบอกว่าเป็นคลาสปกติ
AbstractClass*	ชื่อคลาสเป็นคำเอียงหนา บ่งบอกว่าเป็นคลาสไม่สมบูรณ์
<<Interface>>	บ่งบอกว่าเป็นอินเตอร์เฟส
←	บ่งบอกการสืบทอด extends
◁	บ่งบอกการ implements
◇	บ่งบอกการเป็นส่วนหนึ่ง composite
methodName() *	ตัวเอียง บ่งบอกว่าเป็นเมธอดไม่สมบูรณ์
attribute หรือ methodName()	ตัวหนา บ่งบอกความเป็น final
attribute หรือ methodName()	ตัวขีดเส้นใต้ บ่งบอกความเป็น static

XII. Constructor

คือ เมธอดที่มีชื่อเหมือนกับคลาส ซึ่งใช้กำหนดค่าเริ่มต้นให้กับแอททริบิวต์หรือการทำงานเบื้องต้น

XIII. This / Super

this	ใช้เพื่อเรียกใช้งานแอททริบิวต์หรือเมธอดภายในคลาส
super	ใช้เพื่อเรียกใช้งานแอททริบิวต์หรือเมธอดของคลาสแม่
this(...)	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ภายในคลาส
super(...)	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ของคลาสแม่

XIV. Keyword "Final"

Class	คลาสอื่นไม่สามารถสืบทอดคลาสนี้ได้
Method	คลาสอื่นไม่สามารถ overridden ได้
Variable	ค่าคงที่, สามารถกำหนดค่าได้แค่ครั้งเดียวเท่านั้น
Attribute	

XV. Abstract Class

คลาสแบบ abstract คือ คลาสที่ไม่สมบูรณ์ คลาสที่มีเมธอดที่ยังไม่สามารถระบุการกระทำ (เมธอด) ให้ชัดเจนได้

- Abstract Method

```
[modifier] abstract <ClassName>{
[modifier] abstract return_type <methodName>();
```

XVI. Interface

```
[modifier] interface InterfaceName(){
    [abstract method()];
```

มีลักษณะคล้ายกับ abstract class แต่จะประกอบด้วยเมธอดที่ยังไม่สมบูรณ์เท่านั้น กำหนดขึ้นมาเพื่อให้

```
[modifier] class ClassName implements InterfaceName {
    methods();
}
```

คลาสอื่นนำไปใช้งาน โดยใช้คีย์เวิร์ด implements โดยมีรูปแบบดังนี้

แอททริบิวต์ของอินเตอร์เฟสจะกำหนดให้เป็น **public static final** และเมธอดของอินเตอร์เฟสจะกำหนดให้เป็น **public abstract**

XVII. ตัวอย่างโค้ดจาก Labs

- Labs 6 — Class seller

```
public class Seller extends Employee {
    public Food sell(Employee e) {
        if (e.getWallet().getBalance() >= Food.getPrice()) {
            this.getWallet().setBalance(this.getWallet().getBalance() - Food.getPrice());
            e.getWallet().setBalance(e.getWallet().getBalance() - Food.getPrice());
            return new Food();
        } else {
            System.out.println("Your money is not enough.");
            return null;
        }
    }
}
```

- Labs 6 — Class Employee Method buyFood

```
public boolean buyFood(Seller s) {
    Food newFood = s.sell(this);
    if (newFood == null) {
        return false;
    }
    this.eat(newFood);
    return true;
}
```

- Labs 6 — Class Employee Method toString
*return เป็น String ไม่ใช่ให้ print

```
public String toString() {
    return String!!!!!!;
}
```

- Labs 6 — Class Programmer Method

```
public void coding(char str) {
    String preText = "Your code is ", errorText = "Error Error Error";
    int currentEnergy = getEnergy(), decreaseHappiness = 30, badEnergy = 30;
    if (this instanceof SeniorProgrammer) {
        preText = "I'm coding about ";
        errorText = "ZzZzZz";
        badEnergy = 10;
    }

    setEnergy(currentEnergy - decreaseHappiness);
    setHappiness(getHappiness() - decreaseHappiness);
    if (currentEnergy >= badEnergy) {
        System.out.println(preText + str);
    } else {
        System.out.println(errorText);
    }
}
```

coding(str) (มีการใช้ instanceof)

- Labs7 — Interface Dieselable

XVIII. Note to yourself.

